



LeetCode 1360 – Number of Days Between Two Dates.

1. Problem Title & Link

Title: LeetCode 1360 – Number of Days Between Two Dates

Link: <https://leetcode.com/problems/number-of-days-between-two-dates/>

2. Problem Statement (Short Summary)

Given two dates in the format **YYYY-MM-DD**, return the **absolute difference in days** between them.

The dates are valid and lie between **1971 and 2100**.

3. Examples (Input → Output)

Example 1

Input

date1 = "2019-06-29"

date2 = "2019-06-30"

Output

1

Explanation

Only one day separates the two dates.

Example 2

Input

date1 = "2020-01-15"

date2 = "2019-12-31"

Output

15

Example 3

Input

date1 = "1971-01-01"

date2 = "1971-01-01"

Output

0

4. Constraints

- Dates are between **1971-01-01** and **2100-12-31**
- Format is always **YYYY-MM-DD**



- Dates are valid
- Need absolute difference
- Leap years must be handled correctly

5. Core Concept (Pattern / Topic)

- Date Simulation
- Calendar Mathematics
- Leap Year Calculation

6. Thought Process (Step-by-Step Explanation)

Brute Force

Start from one date.

Increment one day at a time until reaching the other date.

Count the number of days.

This works but can take thousands of iterations.

Time Complexity:

$O(\text{number of days})$

Optimized Thinking

Instead of moving day by day,

convert each date into

"number of days elapsed since 1971-01-01"

Then simply compute

$\text{abs}(\text{days1} - \text{days2})$

Now each conversion only needs to process at most **130 years**, which is constant.

Intuition

To compute total days:

Total Days =

Days from complete years

+

Days from complete months

+

Current day

Whenever a leap year occurs,

February contributes **29 days instead of 28**.



Why This Works

Every date can be uniquely represented by the number of elapsed days from a fixed reference date.

Once both dates become integers,
their difference is simply the answer.

7. Visual / Intuition Diagram (ASCII)

```
Suppose
Reference

1971-01-01
    |
    |-----> Date1
    |
    |-----> Date2
Convert both into
1971-01-01

↓

0 days

↓

Date1 = 17600 days

↓

Date2 = 17615 days
Difference
17615 - 17600 = 15
```

8. Pseudocode (Language Independent)

```
function convert(date):

    split year, month, day

    total = 0

    for every previous year
        add 365
        if leap year
            add 1

    for every previous month
        add days in month

    if February of leap year
```



```
        add 1

    total += day

    return total

answer = abs(convert(date1) - convert(date2))
```

9. Code Implementation

Python

```
class Solution:
    def daysBetweenDates(self, date1: str, date2: str) -> int:

        days = [31,28,31,30,31,30,31,31,30,31,30,31]

        def leap(year):
            return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)

        def convert(date):
            y, m, d = map(int, date.split("-"))

            total = 0

            for year in range(1971, y):
                total += 365
                if leap(year):
                    total += 1

            for month in range(1, m):
                total += days[month - 1]
                if month == 2 and leap(y):
                    total += 1

            return total + d

        return abs(convert(date1) - convert(date2))
```

Java

```
class Solution {

    private final int[] days = {31,28,31,30,31,30,31,31,30,31,30,31};

    public int daysBetweenDates(String date1, String date2) {
        return Math.abs(convert(date1) - convert(date2));
    }

    private int convert(String date) {
```



```
String[] parts = date.split("-");

int year = Integer.parseInt(parts[0]);
int month = Integer.parseInt(parts[1]);
int day = Integer.parseInt(parts[2]);

int total = 0;

for (int y = 1971; y < year; y++) {
    total += 365;
    if (isLeap(y))
        total++;
}

for (int m = 1; m < month; m++) {
    total += days[m - 1];
    if (m == 2 && isLeap(year))
        total++;
}

return total + day;
}

private boolean isLeap(int year) {
    return year % 400 == 0 || (year % 4 == 0 && year % 100 != 0);
}
}
```

10. Time & Space Complexity

Time Complexity

$O(Y + M)$

where

- $Y \leq 130$ years
- $M \leq 12$ months

Practically,

$O(1)$

Space Complexity

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases

- Forgetting leap years



- Incorrect leap-year rule ($\text{year} \% 100 \neq 0$)
- Not adding February's extra day
- Returning signed difference instead of absolute difference
- Incorrect month indexing

12. Detailed Dry Run

Example

date1 = "2019-06-29"

date2 = "2019-06-30"

Convert Date1

Step	Total
Years 1971–2018	17532
Jan	31
Feb	28
Mar	31
Apr	30
May	31
Day	29
Final	17682

Convert Date2

Step	Total
Same previous days	17682
Day becomes 30	17683

Answer

$$17683 - 17682 = 1$$

13. Common Use Cases (Real-Life / Interview)

- Age calculation
- Hotel booking duration
- Employee attendance systems
- Subscription validity
- Project timeline calculations
- Calendar applications



14. Common Traps (Important!)

- Using incorrect leap-year logic
- Forgetting February's extra day
- Wrong month iteration (month - 1)
- Ignoring absolute difference
- Counting the current day incorrectly

15. Builds To (Related LeetCode Problems)

- LC 1507 – Reformat Date
- LC 1154 – Day of the Year
- LC 1185 – Day of the Week
- LC 2409 – Count Days Spent Together
- LC 1360 – Number of Days Between Two Dates

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Simulate day by day	$O(\text{days})$	$O(1)$	Too slow
Convert to ordinal day count	$O(130)$	$O(1)$	Efficient
Built-in Date Library	$O(1)$	$O(1)$	Easiest but often avoided in interviews

17. Why This Solution Works (Short Intuition)

Each date is converted into its total number of elapsed days from a fixed reference date (1971-01-01). Since both dates use the same reference, subtracting these totals gives the exact number of days between them, including leap years.

18. Variations / Follow-Up Questions

1. Solve using built-in date libraries.
2. Support dates before 1971.
3. Handle arbitrary date ranges (BC/AD).
4. Find the day of the week for a given date.
5. Count business days (excluding weekends).
6. Compute months or years between two dates.
7. Extend the solution to support date-time differences (hours, minutes, seconds).
- 8.